


```

e; end VALUE_IS_IN_CHOICE_S; --|
ure RESOLVE_ARRAY_SUBAGGREGATE (EXP : TREE; COMP_TYPE : TREE; INDEX_LIST : SEQ_TYPE; SCALAR_LIST : in-out SEQ_TYPE; NAMED_OTHERS_OK: Boolean := False
) is
  GENERAL_ASSOC_S : TREE := D (AS_GENERAL_ASSOC_S, EXP); INDEX := HEAD (INDEX_LIST); INDEX_TAIL := TAIL (INDEX_LIST);
  ASSOC_COUNT := Natural := COUNT_AGGREGATE_CHOICES (GENERAL_ASSOC_S); AGGREGATE_ARRAY := AGGREGATE_ARRAY_TYPE (1 .. ASSOC_COUNT); TYPESET
  T := TYPESET_TYPE; NEW_ASSOC_LIST := SEQ_TYPE := (TREE_NIL, TREE_NIL); CHOICE := TREE; INDEX_TYPE := TREE; POSITIONAL_SEEN := Boolean := F
alse; NAMED_SEEN := Boolean := False; OTHERS_SEEN := Boolean := False; IS_RANGE := Boolean; begin
  D (SM_DISCRETE_RANGE, EXP, TREE_VOID); -- SPREAD AGGREGATE INTO ARRAY SPREAD_ASSOC_S (GENERAL_ASSOC_S, AGGREGATE_ARRAY); -- RES
OLVE SUBEXPRESSIONS if IS_EMPTY (INDEX_TAIL) then for I in AGGREGATE_ARRAY' RANGE loop if AGGREGATE_ARRAY (I).FIRST = I then TYPESET
0130 := AGGREGATE_ARRAY (I).TYPESET; REQUIRE_TYPE (GET_BASE_TYPE (COMP_TYPE), AGGREGATE_ARRAY (I).EXP, TYPESET); if AGGREGATE_ARRAY (I).EXP := RESOLVE_EXP
OR AGGREGATE (AGGREGATE_ARRAY (I).EXP, COMP_TYPE, NAMED_OTHERS_OK = True); end if; else for I in AGGREGATE_ARRAY' RA
NGE loop if AGGREGATE_ARRAY (I).FIRST = I then if AGGREGATE_ARRAY (I).EXP.TY = DN_AGGREGATE then RESOLVE_ARRAY_SUBAGGREGATE (AGGREGATE_ARR
AY (I).EXP, COMP_TYPE, INDEX_TAIL, SCALAR_LIST, NAMED_OTHERS_OK); else if AGGREGATE_ARRAY (I).EXP.TY = DN_STRING_LITERAL and then IS_EMPTY (TAIL (INDEX
_TAIL)) and then (IS_CHARACTER_TYPE (GET_BASE_TYPE (COMP_TYPE)) or else COMP_TYPE = TREE_VOID) then RESOLVE_STRING_SUBAGGREGATE (AGGREGATE_ARRAY (I
).EXP, COMP_TYPE, HEAD (INDEX_TAIL), SCALAR_LIST); else ERROR (D (LX_SRCPOS, AGGREGATE_ARRAY (I).EXP), "INVALID FORM FOR SUBAGGREGATE"); EVAL_E
XP_TYPES (AGGREGATE_ARRAY (I).EXP, TYPESET); AGGREGATE_ARRAY (I).EXP := RESOLVE_EXP (AGGREGATE_ARRAY (I).EXP, TREE_VOID); end if;
end loop; end if; -- CONSTRUCT NEW ASSOC LIST for I in AGGREGATE_ARRAY' RANGE loop if AGGREGATE_ARRAY (I).FIRST = I then
if AGGREGATE_ARRAY (I).CHOICE := TREE_VOID then AGGREGATE_ARRAY (I).ASSOC := AGGREGATE_ARRAY (I).EXP; POSITIONAL_SEEN := True; el
se D (AS_EXP, AGGREGATE_ARRAY (I).ASSOC, AGGREGATE_ARRAY (I).EXP); if AGGREGATE_ARRAY (I).CHOICE.TY = DN_CHOICE_OTHERS then OTHERS_SEEN := True; el
0140 else NAMED_SEEN := True; end if; end if; NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, AGGREGATE_ARRAY (I).ASSOC); end if;
end loop; -- REPLACE LIST IN GENERAL_ASSOC_S WITH RESOLVED LIST if POSITIONAL_SEEN then LIST (GENERAL_ASSOC_S, NEW_ASSOC_LIST);
end if; -- IF A NAMED ASSOCIATION WAS SEEN if NAMED_SEEN then if POSITIONAL_SEEN then ERROR (D (LX_SRCPOS, EXP), "POSITIONA
L AND NAMED ASSOCIATIONS NOT ALLOWED"); else if not NAMED_OTHERS_OK and OTHERS_SEEN then ERROR (D (LX_SRCPOS, EXP), "POSITIONA
T ALLOWED WITH OTHERS"); end if; -- EVALUATE CHOICES if INDEX.TY = DN_INDEX then -- (NOTE. ANON INDEX BASE TYPE MAY HAVE VOID
EXPRESSION) INDEX_TYPE := D (SM_TYPE_SPEC, INDEX); else INDEX_TYPE := INDEX; end if; INDEX_TYPE := GET_BASE_TYPE (I
NDEX_TYPE); for I in AGGREGATE_ARRAY' RANGE loop CHOICE := AGGREGATE_ARRAY (I).CHOICE; case CHOICE.TY is when DN_CHOICE_EXP =>
EVAL_EXP_SUBTYPE_TYPES (D (AS_EXP, CHOICE), TYPESET, IS_RANGE); REQUIRE_TYPE (INDEX_TYPE, D (AS_EXP, CHOICE), TYPESET); if IS_RANGE then
declare NEW_CHOICE : TREE := MAKE (DN_CHOICE_RANGE); begin
D (AS_DISCRETE_RANGE, NEW_CHOICE, RESOLVE_DISCRETE_RANGE (D (AS_EXP, CH
OICE), GET_THE_TYPE (TYPESET))); D (LX_SRCPOS, NEW_CHOICE, D (LX_SRCPOS, CHOICE); CHOICE := NEW_CHOICE; end; AGGREGATE_ARRAY
0150 (I).CHOICE := CHOICE; if AGGREGATE_ARRAY (I).FIRST = I and then IS_EMPTY (TAIL (LIST (D (AS_CHOICE_S, AGGREGATE_ARRAY (I).ASSOC)))) then
-- REPLACE SINGLETON LIST LIST (D (AS_CHOICE_S, AGGREGATE_ARRAY (I).ASSOC), SINGLETON (CHOICE)); end if; else D (AS_EXP, CHOICE
, RESOLVE_EXP (D (AS_EXP, CHOICE), TYPESET)); end if; when DN_CHOICE_RANGE => EVAL_DISCRETE_RANGE (D (AS_DISCRETE_RANGE, CHOICE), TYPESET);
REQUIRE_TYPE (INDEX_TYPE, D (AS_DISCRETE_RANGE, CHOICE), TYPESET); D (AS_DISCRETE_RANGE, CHOICE, RESOLVE_DISCRETE_RANGE (D (AS_DISCRETE_RANGE, CHOI
CE), GET_THE_TYPE (TYPESET))); when DN_CHOICE_OTHERS => null; when others => null; end case; end loop; MAKE_NORMALIZED_LIS
T (AGGREGATE_ARRAY, NEW_ASSOC_LIST); GENERAL_ASSOC_S := COPY_NODE (GENERAL_ASSOC_S); LIST (GENERAL_ASSOC_S, NEW_ASSOC_LIST);
D (SM_NORMALIZED_COMP_S, EXP, GENERAL_ASSOC_S); end RESOLVE_ARRAY_SUBAGGREGATE;
--|
out SEQ_TYPE) is begin
D (SM_EXP_TYPE, EXP, TREE_VOID); D (SM_DISCRETE_RANGE, EXP, TREE_VOID);
--| procedure MAKE_NORMALIZED_LIST (AGGRE
0160 GATE_ARRAY := in out AGGREGATE_ARRAY_TYPE; NORMALIZED_LIST : out SEQ_TYPE) is
-- MAKES NORMALIZED LIST FOR ARRAY AGGREGATE AGGREGATE_ITEM :=
AGGREGATE_ITEM_TYPE; NEW_NORMALIZED_LIST : SEQ_TYPE := (TREE_NIL, TREE_NIL); NON_STATIC_SEEN := Boolean := False; CHOICE := TRE
E; RANGE_NODE := TREE; begin
NORMALIZED_LIST := (TREE_NIL, TREE_NIL); -- FOR EACH CHOICE for II in AGGREGATE_ARRAY' RANGE loop
if II < AGGREGATE_ARRAY' LAST and then AGGREGATE_ARRAY (II + 1).FIRST = II then -- PREMIER MAIS PAS SEUL declare CHOICE_S :
TREE := MAKE (DN_CHOICE_S); NAMED := TREE := MAKE (DN_NAMED); begin
LIST (CHOICE_S, SINGLETON (AGGREGATE_ARRAY (II).CHOICE)); D (LX_SRCPOS
S, CHOICE_S, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE)); D (AS_EXP, NAMED, D (AS_EXP, AGGREGATE_ARRAY (II).ASSOC)); D (AS_CHOICE_S, NAMED, CHOI
CE_S); D (LX_SRCPOS, NAMED, D (LX_SRCPOS, AGGREGATE_ARRAY (II).ASSOC)); AGGREGATE_ARRAY (II).ASSOC := NAMED; end;
if PAS PRE
MIER AGGREGATE_ARRAY (II).CHOICE.TY = DN_CHOICE_RANGE and then D (AS_DISCRETE_RANGE, AGGREGATE_ARRAY (II).CHOICE).TY = DN_DISCRETE_SUBTYPE the
n AGGREGATE_ARRAY (II).ASSOC := COPY_NODE (AGGREGATE_ARRAY (II).ASSOC); -- POURRAIT CHANGER DE TYPE DISCRET RANGE end if; else
0170 -- PAS LE PREMIER CHOICE declare CHOICE_S : TREE := MAKE (DN_CHOICE_S); NAMED := TREE := MAKE (DN_NAMED); begin
LIST (CHOICE_S, SINGLETON (AGGREGATE_ARRAY (II).CHOICE)); D (LX_SRCPOS, CHOICE_S, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE)); D (AS_EXP, NAMED, D (AS_EXP, AGGR
EGATE_ARRAY (AGGREGATE_ARRAY (II).FIRST).ASSOC)); D (AS_CHOICE_S, NAMED, CHOICE_S); D (LX_SRCPOS, NAMED, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE));
AGGREGATE_ARRAY (II).ASSOC := NAMED; end; end if; -- REUSE EXP AS VALUE OF STATIC CHOICE -- COMPUTE FIRST STATIC VALUE F
OR CHOICE CHOICE := AGGREGATE_ARRAY (II).CHOICE; if CHOICE.TY = DN_CHOICE_EXP then -- (IT'S A CHOICE_EXP) AGGREGATE_ARRAY (II
).EXP := GET_STATIC_VALUE (D (AS_EXP, CHOICE)); else if CHOICE.TY = DN_CHOICE_RANGE then -- (IT'S A CHOICE_RANGE) AGGREGATE_ARRAY (II
).EXP := TREE_VOID; RANGE_NODE := D (AS_DISCRETE_RANGE, CHOICE); if RANGE_NODE.TY = DN_DISCRETE_SUBTYPE then -- (RANGE-GIVEN A
S DISCRETE SUBTYPE -- FIND RANGE) RANGE_NODE := D (AS_SUBTYPE_INDICATION, RANGE_NODE); if D (AS_CONSTRAINT, RANGE_NODE) /= TREE_VOID then RANGE_N
ODE := D (AS_CONSTRAINT, RANGE_NODE); else RANGE_NODE := D (AS_NAME, RANGE_NODE); if RANGE_NODE.TY = DN_SELECTED then RANGE_NODE := D (AS
DESIGNATOR, RANGE_NODE); end if; RANGE_NODE := D (SM_DEFN, RANGE_NODE); if RANGE_NODE.TY in CLASS_TYPE_NAME then RANGE_NODE := D (SM_TYP
E_SPEC, RANGE_NODE); end if; if RANGE_NODE.TY in CLASS_SCALAR then RANGE_NODE := D (SM_RANGE, RANGE_NODE); end if; end if; if RANGE_N
ODE.TY = DN_RANGE and then GET_STATIC_VALUE (D (AS_EXP1, RANGE_NODE)) /= TREE_VOID and then GET_STATIC_VALUE (D (AS_EXP2, RANGE_NODE)) /= TREE_VOID th
en CHOICE := COPY_NODE (CHOICE); -- LE SOUS TYPE DISCRET EST-STATIQUE, REMPLACER-PAR UNE RANGE declare CHOICE_S : TREE := MAKE (DN_CHOIC
E_S); begin
LIST (CHOICE_S, SINGLETON (CHOICE)); D (LX_SRCPOS, CHOICE_S, TREE_VOID); D (AS_CHOICE_S, AGGREGATE_ARRAY (II).ASSOC, CHO
ICE_S); end; D (AS_DISCRETE_RANGE, CHOICE, RANGE_NODE); end if; end if; -- GET STATIC VALUE FOR FIRST ELEMENT OF RANGE if
RANGE_NODE.TY = DN_RANGE then AGGREGATE_ARRAY (II).EXP := GET_STATIC_VALUE (D (AS_EXP1, RANGE_NODE)); else AGGREGATE_ARRAY (II).EXP := TREE
VOID; end if; else if CHOICE.TY = DN_CHOICE_OTHERS then AGGREGATE_ARRAY (II).EXP := TREE_VOID; else -- (NOT-CHOICE_ANYT
HING; ERROR MUST HAVE BEEN REPORTED) return; end if; -- CHECK FOR ILLEGAL NON-STATIC if AGGREGATE_ARRAY (II).EXP = TREE_VOI
D and CHOICE.TY = DN_CHOICE_OTHERS and AGGREGATE_ARRAY' LENGTH > 1 then NON_STATIC_SEEN := True; ERROR (D (LX_SRCPOS, CHOICE), "CHOICE
MUST BE STATIC"); end if; end loop; if NON_STATIC_SEEN then return; end if; -- SORT THE ENTRIES for II in AGGRE
0190 GATE_ARRAY' FIRST + 1 .. AGGREGATE_ARRAY' LAST loop exit when AGGREGATE_ARRAY (JJ).EXP = TREE_VOID; -- OTHERS for JJ in reverse AGGREG
ATE_ARRAY' FIRST .. II - 1 loop exit when UARITH "<" = (AGGREGATE_ARRAY (JJ).EXP, AGGREGATE_ARRAY (JJ + 1).EXP); AGGREGATE_ITEM := AG
GREGATE_ARRAY (JJ); AGGREGATE_ARRAY (JJ) := AGGREGATE_ARRAY (JJ + 1); AGGREGATE_ITEM := AGGREGATE_ITEM; end loop;
end loop; -- CONSTRUCT THE NEW LIST for II in AGGREGATE_ARRAY' RANGE loop NEW_NORMALIZED_LIST := APPEND (NEW_NORMALIZED_LIST, AGGR
EGATE_ARRAY (II).ASSOC); end loop; NORMALIZED_LIST := NEW_NORMALIZED_LIST; end MAKE_NORMALIZED_LIST;
end AGGREGATE;

```